

frontend/hooks | TypeScript

```
/**
 * useFusionEngine.ts
 */
 * Central orchestration hook – combines all three signal channels into a
 * single CognitiveAssessment that drives UI state and AI responses.
 */
 * Signal channels:
 * useRPPG → rppgScore (visual, weight 0.3 standard / 0.1 adaptive)
 * useVocalJitter → jitterScore (audio, weight 0.5 always)
 * semanticScore → prop from speech recognition (semantic, weight 0.2/0.4)
 */
 * Adaptive weight redistribution:
 * When rPPG signalConfidence < 60%, computeFusionScore() automatically
 * switches to adaptive weights (J:0.5, V:0.1, I:0.4) to compensate for
 * degraded visual channel. This is documented in docs/architecture.md §2.2.
 */
 * Output feeds:
 * CognitiveStateMachine → state (CALM / LOAD / CRITICAL)
 * VpgGraph → waveform color + stress level
 * AI intervention box → Groq prompt context
 * useStateMemory → intervention logging
 */

import { useEffect, useRef, useState } from 'react';
import {
computeFusionScore,
CognitiveStateMachine,
CognitiveAssessment,
computeSignalConfidence,
} from '../utils/stressEngine';
import { useRPPG } from './useRPPG';
import { useVocalJitter } from './useVocalJitter';

// ■■■ Types
interface UseFusionEngineProps {
/** Semantic intent score 0-100, driven by hinglishParser via speech events */
semanticScore: number;
}

interface UseFusionEngineReturn {
/** Full cognitive assessment – score, state, transition flag, prev state */
assessment: CognitiveAssessment;
/** Overall signal confidence 0-100 – drives adaptive weight redistribution */
signalConfidence: number;
/** True during the 3-second calibration phase on first load */
isCalibrating: boolean;
/** Whether adaptive weights are currently active (rPPG confidence < 60%) */
isAdaptiveMode: boolean;
/** Raw sub-scores from each channel – for debug display in the UI */
channels: {
visual: number; // rPPG variance score
audio: number; // vocal jitter score
semantic: number; // Hinglish intent score
};
/** Refs to pass to the hidden <video> and <canvas> elements in the layout */
videoRef: React.RefObject<HTMLVideoElement>;
canvasRef: React.RefObject<HTMLCanvasElement>;
}

// ■■■ Constants
/** rPPG confidence below this triggers adaptive weight redistribution */
const ADAPTIVE_CONFIDENCE_THRESHOLD = 60;
```

